LEMBAR KERJA 08

Code as a Trigger: Otomatisasi Test, Retraining, Evaluasi, dan Registrasi Model

Deteksi Anomali Cuaca Open-Meteo

Disusun oleh

Muhammad Ghazy Humaidi
245150200111071

Mata Kuliah MLOps
Dosen Rizal Setya Perdana, S.Kom., M.Kom., Ph.D.

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA

2026

1 Ringkasan Luaran

LK08 membangun alur kerja GitHub Actions yang berjalan setiap kali kode masuk ke branch **main** atau ada pull request ke main. Satu perubahan kode memicu rangkaian tahap secara berurutan: pemulihan data dari DVC, pengujian pytest, training ulang melalui **train.py**, evaluasi terhadap ambang performa LK01, lalu registrasi otomatis ke MLflow Model Registry dengan stage **Staging**. Jika satu tahap gagal, tahap sesudahnya tidak dijalankan.

Luaran	Lokasi / bukti
File konfigurasi	.github/workflows/mlops-automation.yaml (lampiran A)
Kebijakan gate	configs/lk08.yaml; logika di src/automation.py dan src/validation.py
Repositori	https://github.com/ghazthiskc19/MLOps-AnomaliCuacaOpenMeteo
Log eksekusi (commit awal)	https://github.com/ghazthiskc19/MLOps-AnomaliCuacaOpenMeteo/actions/runs/42501642880
Log eksekusi (simulasi perubahan)	https://github.com/ghazthiskc19/MLOps-AnomaliCuacaOpenMeteo/actions/runs/42502026204

Kedua run di atas dipicu oleh event **push** ke main tanpa langkah manual di antara tahap. Log baris per baris di GitHub hanya dapat dibuka setelah login. Karena itu ringkasan hasil gate dan versi registry juga dicetak sebagai *annotation* yang tampil di halaman run untuk siapa pun.

2 Rancangan Pipeline

Tahap	Step workflow	Fungsi
Pemicu	on: push / pull_request ke main	Setiap perubahan kode langsung menjalankan pipeline; concurrency membatalkan run lama pada ref yang sama
Persiapan	checkout (fetch-depth 0), setup-python 3.12.3, pip install + pip check	Lingkungan bersih di runner Ubuntu; histori dan tag Git ikut diambil untuk verifikasi pointer DVC
Data	dvc pull -r r2	Mengambil snapshot dataset-v3.0.0 dan dataset-v4.0.0 dari Cloudflare R2 memakai GitHub Secrets
Tahap 1	python -m pytest tests	Menguji preprocessing, fitur, skenario, validasi, dan registrasi
Tahap 2	python train.py --config configs/lk08.yaml	Training ulang Random Forest pada data DVC v4; hanya menghasilkan kandidat
Tahap 3	scripts/evaluate_candidate.py	Menghitung ulang prediksi dan metrik, keluar dengan exit code 1 bila gate gagal
Tahap 4	scripts/register_candidate.py	Evaluasi ulang, registrasi ke MLflow, transisi ke Staging, lalu verifikasi stage
Arsip	upload-artifact (if: always())	Paket model, metrik, dan store MLflow disimpan 30 hari, termasuk saat gate menolak

3 Kriteria Kelulusan Otomatis

Laporan LK01 (bagian 5.1) menetapkan target **F1-Score lebih dari 85%**. Nilai ini dipakai sebagai syarat utama dan dibandingkan secara ketat (lebih besar, bukan sama dengan). Syarat lain ditambahkan pada

LK08 agar model tidak lolos hanya karena satu angka rata-rata terlihat bagus.

Syarat	Nilai	Alasan
Macro F1 tiga kelas	> 0.85	Ambang LK01; macro agar kelas minoritas tidak tertutup kelas normal
Support minimum per kelas	>= 5 data	Kelas tanpa data uji dianggap belum terbukti, bukan dianggap lolos
Recall hardware fault	>= 0.85	Fault sensor yang lolos tidak terdeteksi adalah kegagalan paling mahal
False alarm pada data normal	<= 0.05	Alarm palsu yang sering membuat operator mengabaikan sistem
Agreement per jenis skenario	>= 0.8 (23 jenis)	Setiap jenis gangguan diuji terpisah, bukan hanya total

Data uji terdiri dari dua bagian: holdout temporal dari data asli v4 (134 baris, 27 Oktober sampai 1 November 2026) dan uji skenario terpisah (184 event, seed dan partisi berbeda dari training). Holdout temporal tidak memiliki satu pun kejadian extreme weather (kelas 2). Jika dinilai sendiri, macro F1 tiga kelas pasti tertahan di 0,667 berapa pun kualitas modelnya, karena F1 kelas 2 dihitung nol.

Karena itu coverage dan macro F1 dinilai pada **gabungan** kedua bagian, sedangkan holdout temporal tetap wajib memenuhi recall fault dan batas false alarm secara mandiri. Perlu dicatat dengan jujur: aturan gabungan ini ditetapkan setelah pemeriksaan lokal menunjukkan gate ketat selalu gagal. Ambang 0,85 tidak diturunkan, tetapi sumber bukti kelas 2 berasal dari data simulasi, bukan pengamatan lapangan.

Hasil gate pada run simulasi (annotation GitHub):

combined macro F1=1.0000 (threshold > 0.85), support=134/120/64, fault recall=1.0000, false alarm=0.0000, scenario kinds passed=23/23

Hasil registrasi:

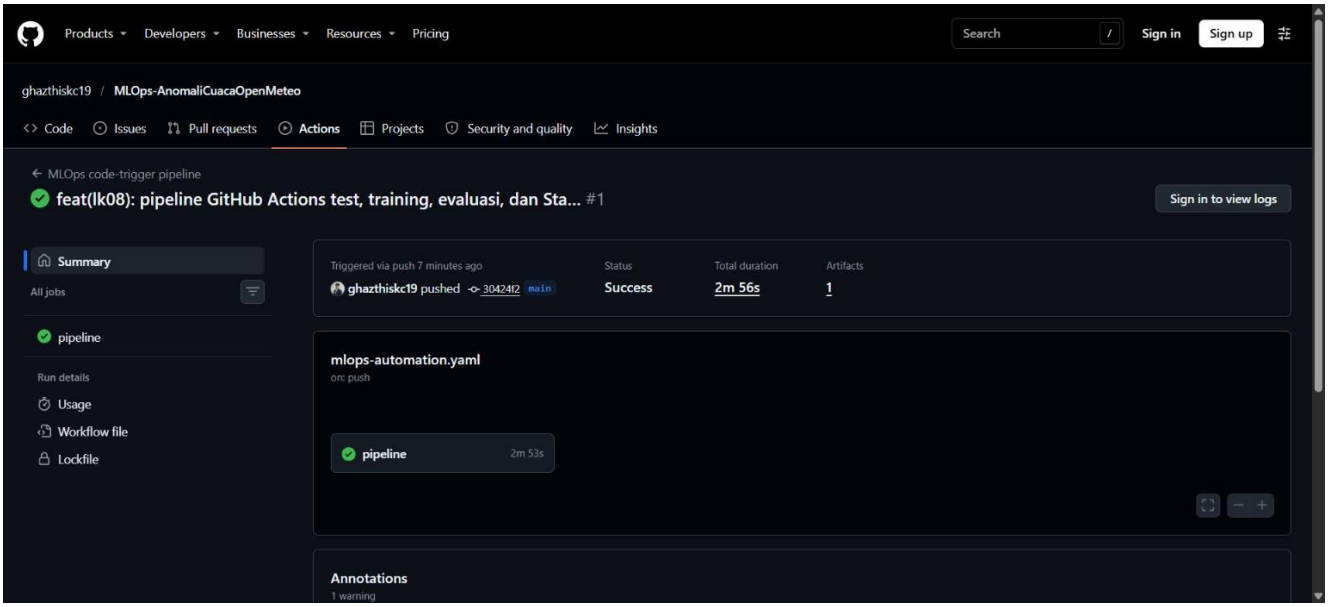
WeatherAnomalyGuardedRF version 1 stage=Staging run_id=405d241aac9747b1b4c7d11f87dd77d4

4 Bukti Eksekusi Otomatis

Run 1: Commit awal pipeline

Commit **30424f2** "feat(lk08): pipeline GitHub Actions test, training, evaluasi, dan Staging" | event push ke main | hasil **success** | durasi job 2 mnt 53 dtk
https://github.com/ghazthiskc19/MLOps-AnomaliCuacaOpenMeteo/actions/runs/42501642880

#	Step	Hasil	Durasi
2	Run actions/checkout@v4	success	3 dtk
3	Run actions/setup-python@v5	success	9 dtk
4	Install and verify dependencies	success	1 mnt 12 dtk
5	Pull frozen DVC snapshots	success	4 dtk
6	Automated testing	success	23 dtk
7	Automated training (candidate only)	success	49 dtk
8	Model evaluation and coverage validation	success	3 dtk
9	Register validated candidate in Staging	success	4 dtk
10	Preserve results even when gate rejects candidate	success	1 dtk



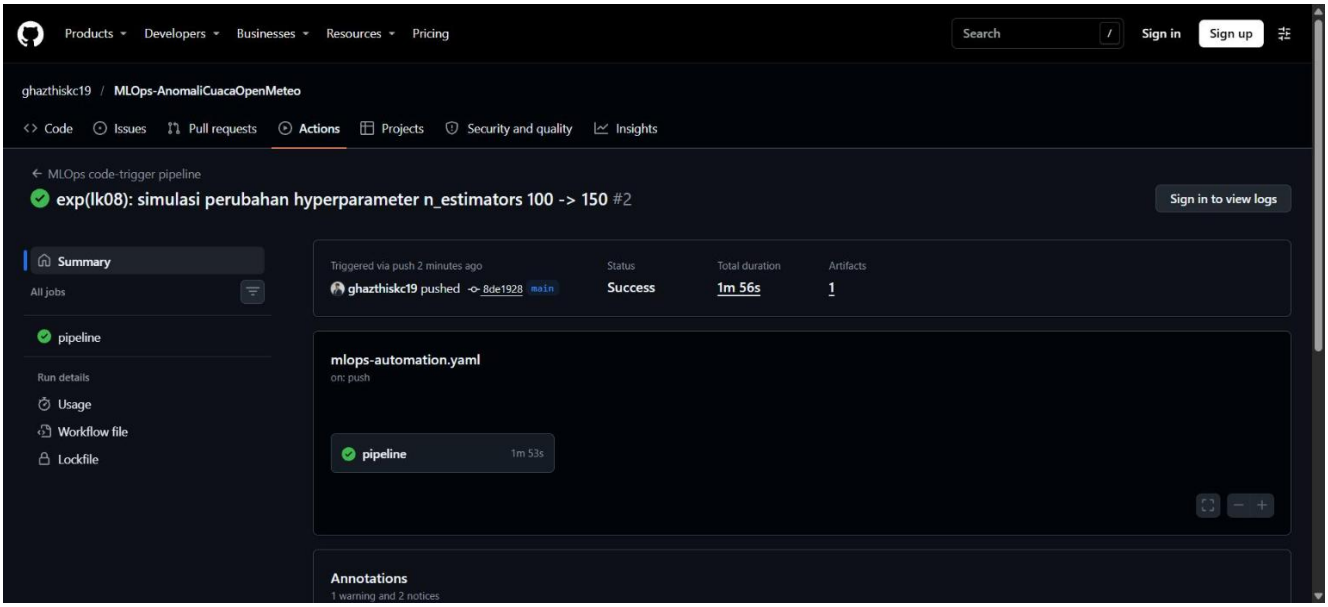
Gambar 1. Ringkasan run #1 (commit awal pipeline) di GitHub Actions: dipicu push, status Success.

Run 2: Simulasi perubahan kode

Commit **8de1928** "exp(lk08): simulasi perubahan hyperparameter n_estimators 100 -> 150" | event push ke main | hasil **success** | durasi job 1 mnt 53 dtk
https://github.com/ghazthiskc19/MLOps-AnomaliCuacaOpenMeteo/actions/runs/42502026204

#	Step	Hasil	Durasi
2	Run actions/checkout@v4	success	2 dtk
3	Run actions/setup-python@v5	success	12 dtk
4	Install and verify dependencies	success	37 dtk

#	Step	Hasil	Durasi
5	Pull frozen DVC snapshots	success	3 dtk
6	Automated testing	success	16 dtk
7	Automated training (candidate only)	success	33 dtk
8	Model evaluation and coverage validation	success	2 dtk
9	Register validated candidate in Staging	success	3 dtk
10	Preserve results even when gate rejects candidate	success	1 dtk



Gambar 2. Ringkasan run #2 (simulasi perubahan kode) di GitHub Actions: dipicu push, status Success.

5 Simulasi Perubahan Kode

Perubahan yang disimulasikan adalah hyperparameter: **n_estimators** pada ketiga konfigurasi kandidat Random Forest (src/automation.py) dinaikkan dari 100 menjadi 150. Commit 8de1928 di-push ke main. Tidak ada perintah yang dijalankan secara manual sesudahnya. GitHub Actions membuat run #2 dan menjalankan test, training ulang, evaluasi, serta registrasi secara berantai dengan hasil **success**.

File yang berubah pada commit tersebut: scripts/collect_lk08_actions_evidence.py, scripts/evaluate_candidate.py, scripts/register_candidate.py, src/automation.py. Commit yang sama juga menambahkan baris annotation pada dua script CLI agar metrik gate dan versi registry terbaca tanpa login. Logika evaluasi tidak berubah.

Store MLflow pada pipeline ini dibuat baru di setiap run (SQLite di runner), lalu diunggah sebagai artifact. Nomor versi registry karena itu dimulai dari 1 pada setiap run dan tidak menunjukkan urutan antar-run. Registry terpusat yang persisten, misalnya server MLflow di VPS, belum dapat dijangkau dari runner GitHub karena VPS hanya terbuka dari jaringan UB.

6 Refleksi: Pemicu Otomatis dan Silent Failures

Silent failure adalah kegagalan yang tidak memunculkan error. Pipeline tetap selesai, model tetap menghasilkan prediksi, dan dasbor tetap hijau, padahal keputusan yang dihasilkan sudah salah. Kegagalan seperti ini berbahaya justru karena tidak ada yang memeriksanya. Proyek ini sendiri memberi beberapa contoh nyata.

Metrik yang menipu. Pada LK07, model mencapai accuracy 0,97 pada holdout v4. Angka itu tampak sangat baik, tetapi macro F1-nya hanya 0,57 karena data uji tidak memuat extreme weather sama sekali. Kalau keputusan promosi diambil manual dengan melihat accuracy, model akan naik ke tahap berikutnya tanpa pernah diuji pada kelas tersebut. Gate otomatis memaksa pemeriksaan yang sama setiap kali: kelas tanpa data dianggap belum terbukti, dan alasan penolakan dicatat (insufficient_coverage, failed_performance) alih-alih disembunyikan dalam satu skor.

Data yang bergeser diam-diam. Sebelum LK06, file raw lokal ternyata berbeda dari pointer DVC (720 baris lokal dibanding 936 baris versi v2). Training tetap berjalan, hanya saja dengan data yang salah. Pipeline LK08 selalu memulihkan data dari remote DVC dan memverifikasi hash MD5 serta tag Git sebelum training. Jika data tidak sesuai, run berhenti dengan error yang terlihat.

Lingkungan yang hanya benar di laptop. Ekspor model memakai skops dan kode memakai PyYAML, tetapi keduanya sempat tidak tercantum di requirements.txt karena kebetulan sudah terpasang secara transitif. Di laptop semua lancar. Runner GitHub selalu memasang dependency dari nol, sehingga dependency yang lupa dicatat langsung membuat build gagal, bukan muncul berminggu-minggu kemudian di server produksi.

Langkah yang dilewati. Pada alur manual, urutan test, training, evaluasi, dan registrasi bergantung pada disiplin orang yang menjalankannya. Satu langkah yang terlewat cukup untuk meloloskan model yang belum diuji. Dengan workflow, urutan itu menjadi kode: registrasi tidak bisa terjadi bila evaluasi gagal, dan register_candidate.py mengevaluasi ulang serta mencocokkan hash artefak sebelum menulis ke registry. Model yang pernah gagal tidak dapat didaftarkan hanya dengan menjalankan perintah registrasi.

Skalabilitas. Pemeriksaan manual mungkin masih sanggup dilakukan untuk satu model dan satu perubahan per minggu. Begitu ada banyak kontributor, perubahan harian, dan data yang terus bertambah dari Airflow, pemeriksaan manual menjadi titik paling lemah sekaligus paling lambat. Otomatisasi membuat biaya setiap

pemeriksaan hampir nol sehingga pemeriksaan bisa dilakukan pada setiap commit. Setiap run juga meninggalkan jejak yang bisa diaudit: commit, hash data, metrik, keputusan gate, dan versi model. Dalam arti itu, otomatisasi adalah tulang punggung AI yang dapat diskalakan. Tanpanya, kualitas model bergantung pada ingatan dan ketelitian orang per orang.

7 Keterbatasan

Keterbatasan	Dampak
Kelas 2 hanya terwakili oleh skenario simulasi	Kelulusan gate menunjukkan kesesuaian dengan aturan label, bukan validasi lapangan
Aturan gate gabungan ditetapkan setelah melihat hasil lokal	Ambang tetap 0,85, tetapi keputusan ini perlu diuji ulang saat snapshot baru memiliki extreme weather alami
Store MLflow per-run di runner	Versi Staging tersimpan sebagai artifact run, bukan pada registry terpusat
API stage MLflow sudah deprecated	Stage dipakai karena diminta tugas; migrasi ke alias perlu dilakukan kemudian
Push langsung ke main	Pull request tidak dibuat karena keterbatasan akses saat pengerjaan; trigger PR tetap terdefinisi di workflow
PR dari fork tidak menerima secrets	Pipeline pada fork akan berhenti di tahap pemulihan DVC dengan pesan error eksplisit

Lampiran A. .github/workflows/mlops-automation.yaml

```
name: MLOps code-trigger pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

concurrency:
  group: mlops-${{ github.event.pull_request.number || github.ref }}
  cancel-in-progress: true

jobs:
  pipeline:
    runs-on: ubuntu-24.04
    timeout-minutes: 30
    env:
      MPLBACKEND: Agg
      PYTHONUTF8: '1'
      MLFLOW_ENABLE_ARTIFACTS_PROGRESS_BAR: 'false'
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12.3'
          cache: pip
      - name: Install and verify dependencies
        run: |
          python -m pip install -r requirements.txt
          python -m pip check
      - name: Pull frozen DVC snapshots
        env:
          AWS_ACCESS_KEY_ID: ${{ secrets.R2_ACCESS_KEY_ID }}
          AWS_SECRET_ACCESS_KEY: ${{ secrets.R2_SECRET_ACCESS_KEY }}
          AWS_DEFAULT_REGION: auto
        run: |
          if [ -z "$AWS_ACCESS_KEY_ID" ] || [ -z "$AWS_SECRET_ACCESS_KEY" ]; then
            echo '::error::R2 secrets required to restore private DVC data. Fork PRs do not receive reposito
ry secrets.'
            exit 1
          fi
          python -m dvc pull -r r2 data/simulated/weather_faults_s42.csv.dvc data/raw/weather_raw_current.cs
v.dvc data/simulated/weather_lk07_faults_s42.csv.dvc data/raw/weather_lk07_history.csv.dvc
      - name: Automated testing
        run: python -m pytest tests -q --junitxml=artifacts/pytest.xml
      - name: Automated training (candidate only)
        run: python train.py --config configs/lk08.yaml --output-dir artifacts/lk08
      - name: Model evaluation and coverage validation
        run: python scripts/evaluate_candidate.py --candidate-dir artifacts/lk08
      - name: Register validated candidate in Staging
        run: python scripts/register_candidate.py --candidate-dir artifacts/lk08
      - name: Preserve results even when gate rejects candidate
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: lk08-${{ github.run_id }}-${{ github.run_attempt }}
          path: |
            artifacts/
            mlruns/lk08/
          if-no-files-found: warn
          retention-days: 30
```

Referensi

GitHub Actions: events that trigger workflows.
https://docs.github.com/en/actions/writing-workflows/choosing-when-your-workflow-runs/events-that-trigger-workflows

GitHub Actions: workflow commands (annotations). https://docs.github.com/en/actions/writing-workflows/choosing-what-your-workflow-does/workflow-commands-for-github-actions

MLflow Model Registry workflows. https://mlflow.org/docs/latest/ml/model-registry/workflow/

DVC pull. https://dvc.org/doc/command-reference/pull